

ソフトウェア設計及び実験

UI3 (Joy-Conプログラミング)

担当：森田

実験内容

- 目的
 - Joy-Conを扱うプログラミングの習得
 - 大規模コーディングの基本を学習
- 説明内容
 - Joy-Conライブラリの使い方
 - 大規模コーディングについて
- 課題
 - Joy-Conを使うプログラムの作成

Joy-Conプログラミング

Joy-Con

- アナログスティックと11個のボタン
- LED, homeボタンのLED (右のみ)
- 6軸の加速度, ジャイロセンサー
- 振動機能
 - 細かい制御も
- NFCリーダー (右のみ)
- IRカメラ (右のみ)
- Bluetooth
- 充電式

Joy-Con (R)



拡張バッテリー



Joy-Conとゲーム制作（何ができるそうか）

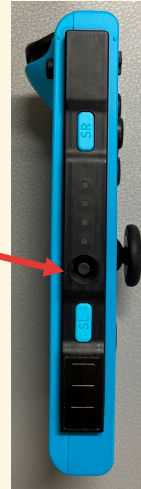
- アナログスティックと11個のボタン
 - 一般的なゲームコントローラーとして
- LED, homeボタンのLED（右のみ）
 - プレイヤーの識別, 状況表示
 - 謎解きゲームのヒントなど
- 6軸の加速度, ジャイロセンサー
 - 振る速さで球の速さを変化させる
 - 傾けた方向にキャラを動かす
- 振動機能
 - ダメージの大きさを表現
 - 簡単な演奏
- IRカメラ（右のみ）
 - OpenCVとの組み合わせで画像処理
- NFCリーダー（右のみ）
 - カードバトルなどで物理カードの読み取り（NFCタグが必要）

PCとの接続

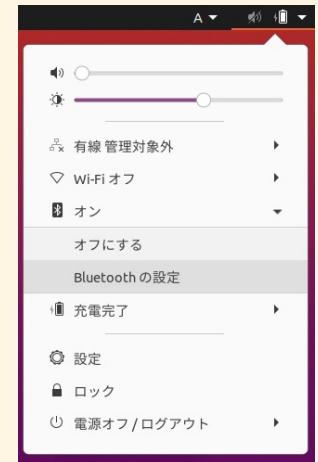
- Bluetooth機器

- 一般的な接続方法と同じ

- シンクロボタンを長押し
LEDが点滅したら



- PCのBluetooth一覧から
Joy-Con (R)を選んで接続



- 接続できてもすぐには使えない

- 制御プログラムの仲介が必要

Joy-Conライブラリ

- Joy-Con制御のためのAPIライブラリ
 - ソフト実験のために制作
 - バグがあるかも
 - 電算室PC，仮想イメージにしか入っていない
- 機能
 - スティック，ボタン，6軸センサーの読み取り
 - 振動機能の制御
 - 4つのLED，ホームボタンLEDの制御
 - IRカメラ画像の読み取り
 - NFCリーダーによるデータ読み取り
- 貸出機器のJoy-Con (R)の他， Joy-Con (L)も

Joy-Conライブラリでのプログラミング

- あらかじめBluetooth接続しておく

1. ヘッダファイルのインクルード

- `#include <joyconlib.h>`

2. Joy-Conのオープン

- (例) `joyconlib_t jc;`
`joycon_open(&jc, JOYCON_R);`

3. Joy-Conの操作

- (例) `joycon_get_state(&jc);`

4. Joy-Conのクローズ

- (例) `joycon_close(&jc);`

• コンパイル方法

- `gcc [ソースファイル.c] -ljoyconlib -lhidapi-hidraw -lm`

ライブラリ関数

- オープン／クローズ
 - `joycon_open()`, `joycon_close()`
- 状態取得
 - `joycon_get_state()`, `joycon_get_button_elapsed()`
- 振動
 - `joycon_rumble()`, `joycon_rumble_raw()`,
`joycon_play_rumble()`
- LED, homeLED
 - `joycon_set_led()`, `joycon_set_homeled()`
- IRカメラ
 - `joycon_enable_ir()`, `joycon_disable_ir()`, `joycon_read_ir()`
- NFCリーダー
 - `joycon_read_nfc()`

ライブラリ関数（状態取得）

- `joycon_get_state()`
 - ボタン，スティック，6軸センサーの状態を取得する
 - `joyconlib_t` 構造体の変数に保存
 - 例) `joycon_get_state(&jc);`
- `joycon_get_button_elapsed()`
 - 特定のボタン（R,ZR,SL,SR,Home）が押されてからの経過時間を10ms単位で取得する
 - `joycon_elapsed`構造体の変数に保存
 - 例)
`joycon_elapsed et;`
`joycon_get_button_elapsed(&jc, &et);`

joyconlib_t 構造体（よく使うもの）

- すべてのライブラリ関数に必要
 - Joy-Conの状態を保持
- 読み取り専用と考える
- 変数を用意して，オープンから同じ変数を使う

```
joyconlib_t jc;
```

```
joycon_open(&jc, JOYCON_R);
```

```
typedef struct {  
    :  
    u8 battery;                バッテリー残量（4段階）  
    joycon_btn button;         ボタンの状態  
    joycon_stick stick;        スティックの値  
    joycon_axis axis[3];       6軸センサーの情報  
    :  
} joyconlib_t;
```

スティック, ボタン

- スティック

- 0~1の
アナログ値

```
typedef struct {  
    float x;  
    float y;  
} joycon_stick;
```

- 中心からの距離が1となる
ような値を取る
- 縦持ちにしたときの上下がy

例) 上に移動させる

```
if( jc.stick.y < 0 ){...}
```

- ボタン

- 押されているときに1,
離されているときに0となる

例) Aボタンが押されているとき

```
if( jc.button.btn.A ){...}
```

```
typedef union {  
    struct {  
        u8 Y;  
        u8 X;  
        u8 B;  
        u8 A;  
        u8 SR_r;    SRボタン  
        u8 SL_r;    SLボタン  
        u8 R;  
        u8 ZR;  
        u8 Plus;    +ボタン  
        u8 RStick;  スティックを押したとき  
        u8 Home;    ホームボタン  
        :  
    } btn;  
    :  
} joycon_btn;
```

6軸センサー

- 3つの加速度センサー

- 単位: m/s^2
- 静止状態が0
 - ただし重力方向は約 $\pm 9.8m/s^2$

- 3つのジャイロセンサー

- 単位: rad/s
- 静止状態が0

- 一度のjoycon_get_state()で5ms間隔3つ分を取得

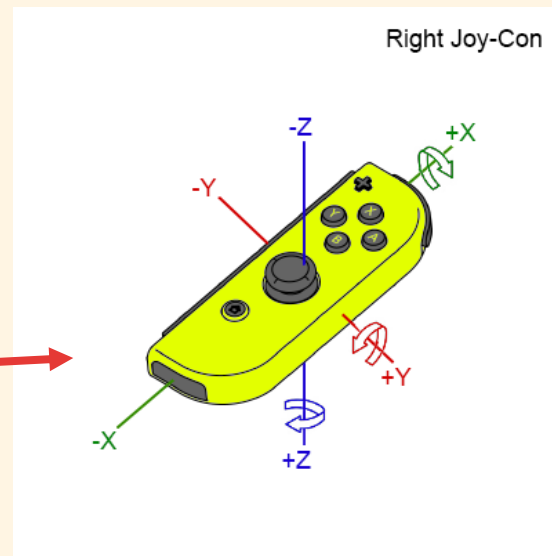
例) z方向の加速度 (最新, 過去5ms, 過去10ms)

- `jc.axis[0].acc_z, jc.axis[1].acc_z, jc.axis[2].acc_z`

- +値, -値の関係は

- 積分で速度や角度も推定

```
typedef struct {  
    float acc_x;    加速度値  
    float acc_y;  
    float acc_z;  
    float gyro_x;   ジャイロ値  
    float gyro_y;  
    float gyro_z;  
} joycon_axis;
```



ライブラリ関数（振動）

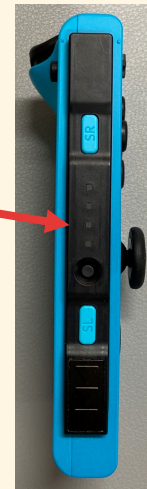
- `joycon_rumble()`
 - Joy-Conを振動させる
 - 強さは0-100, 0でoff, 50くらいでも十分
 - 例) `joycon_rumble(&jc, 40);`
- `joycon_rumble_raw()`
 - Joy-Conを周波数, 振幅を指定して振動させる
 - 詳細はAPI資料参照
- `joycon_play_rumble()`
 - Joy-Conの振動によって演奏させる
 - 詳細はAPI資料参照

ライブラリ関数 (LED, homeLED)

- joycon_set_led()

- Joy-ConのLEDを点灯／点滅させる
- 4つのLEDに対してマクロで指定
- 例) 1を点灯, 3を点滅

```
joycon_set_led(&jc, JOYCON_LED_1_ON | JOYCON_LED_3_BLINK);
```



- joycon_set_homeled()

- Joy-ConのホームボタンのLEDを設定する
- 単純な点灯/点滅はマクロで指定

```
JOYCON_HOMELED_OFF,  
JOYCON_HOMELED_ON, JOYCON_HOMELED_BLINK
```

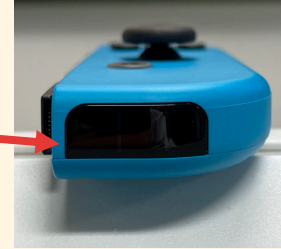
- 例) `joycon_set_homeled(&jc, JOYCON_HOMELED_BLINK);`
- 複雑な点灯パターンも指定できる

- 詳細はAPI資料参照



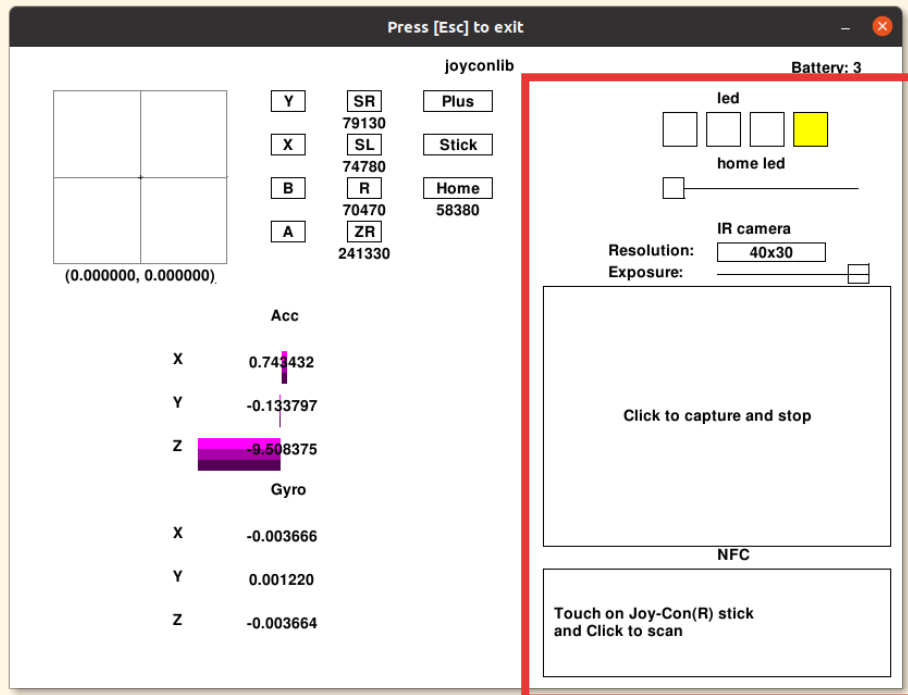
ライブラリ関数 (IRカメラ, NFCリーダー)

- `joycon_enable_ir()`
 - IRカメラの使用を開始する
 - 解像度の指定が必要 (以下の4種)
 - `JOYCON_IR320X240` 320 x 240 ピクセル
 - `JOYCON_IR160X120` 160 x 120 ピクセル
 - `JOYCON_IR80X60` 80 x 60 ピクセル
 - `JOYCON_IR40X30` 40 x 30 ピクセル
- `joycon_disable_ir()`
 - IRカメラの使用を終了する
- `joycon_read_ir()`
 - IRカメラで画像を読み取る
 - 詳細はAPI資料参照
- `joycon_read_nfc()`
 - NFCタグ情報を読み取る
 - スティックがNFCリーダー
 - 詳細はAPI資料参照



サンプルツール

- ソフト実験のページ
(<http://netadm.iss.tokushima-u.ac.jp/soft/>) の
Joy-Conプログラミングサンプル ([dev/joycon_samples.tar.gz](http://dev.joycon_samples.tar.gz))
 - **extool.c** をコンパイルして実行
 - 他のサンプルソースも参考に



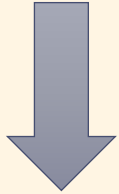
- この部分はマウスで
クリックやドラッグすると
- LED状態変更
 - home LEDの明るさ変更
 - IRカメラ読み取り
 - NFC読み取り

大規模コーディングの基本

プログラムの（基本的）構造

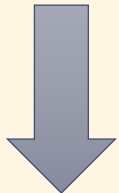
- 初期化処理

- メインの処理に必要なデータの準備
- コストの高い（時間のかかる）処理は、あらかじめ実施



- メイン処理

- プログラムの目的である処理の実施
- ループとなることが多い
 - GUIでは終了指示まで

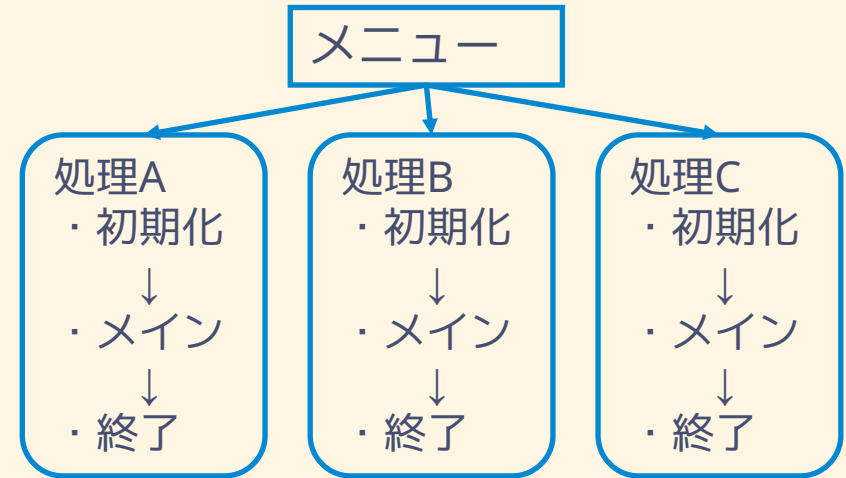


- 終了処理

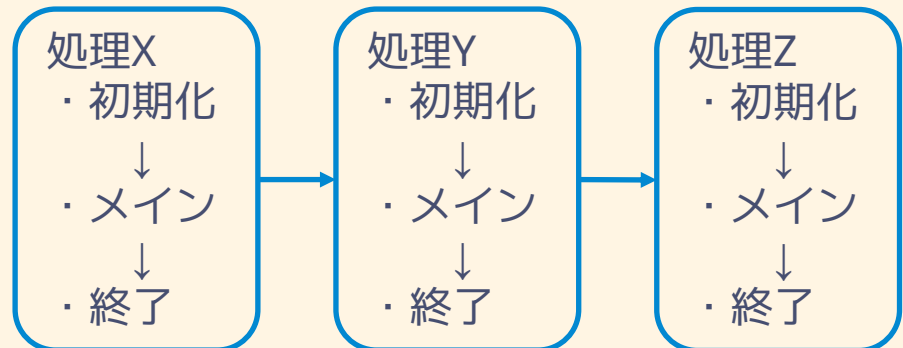
- 初期化処理などで用意したデータの後始末（ファイルを閉じる、メモリ解放、など）

- 複数の処理が混在する場合

- ex) メニューによる選択



- ex) バッチ処理



コストの高い処理とは

- CPU, GPUが内部で計算できる以外の処理
- 主記憶（メモリ）アクセス
 - メモリ確保
 - `malloc()`など、大量のサイズを扱うときは特に
 - `SDL_CreateTexture()`など、Createと名のつく関数は概ねメモリ確保している
 - ライブラリの初期化
 - `SDL_Init()` など、初期化関数内でメモリ確保やファイルアクセスが起きるため
- 2次記憶へのアクセス
 - ファイルの読み込み
 - `fopen()`, `fscanf()` など
 - `IMG_Load()`, `TTF_OpenFont()` など,
 - `IMG_Load()`はファイルを読み込んでサーフェイスを作ってコピーしてる
- 入出力操作
 - キー入力（人の入力を待つため）
 - 通信（ネットワークや無線など）
 - 表示／描画（`printf()`など）
- メイン処理に必要なものは仕方が無いが,
 - そうでなければ初期化処理で実施

ある悪い例

```
// 初期化処理
SDL_Init(SDL_INIT_VIDEO);
IMG_Init(IMG_INIT_PNG);
window = SDL_CreateWindow("some", 100,
100, Window_Width, Window_Height, 0);
render = SDL_CreateRenderer(window, -1, 0);
:
// メイン処理
while (flg) {
    // イベント
    if (SDL_PollEvent(&event)) {
        switch (event.type) {
            :
            case SDL_KEYDOWN:
                // キー入力によって座標変更
                flg = KeyEvent(&event, &rect);
                break;
        }
    }
    // 描画（画像を読み込んで転送）
    surface = IMG_Load("player.png");
    texture =
    SDL_CreateTextureFromSurface( render,
    surface );
    SDL_RenderCopy(render, texture, NULL,
    &rect);
    :
    // 表示
    SDL_RenderPresent(render);
}
:
```

- メイン処理のループ内で画像読み込み
 - 毎回新しく読み込んでメモリ確保している
 - 同じ画像なら一度読み込めばよいはず→初期化処理へ
 - 前に読み込んだ画像が消去されてない
 - メモリに無駄に溜まり続け、いずれ枯渇しエラーとなる
→使い終わったら消去する
(SDL_DestroyTexture(), SDL_FreeSurface())
 - コストの高い処理
 - ループ1回の時間がかかる
→表示間隔が空く（いわゆるフレームレートが落ちる）
- ループ内での転送
 - メイン処理に必要（座標が変更されるため）
 - メモリ間のコピーしているだけ
 - 新たな資源は確保していない
 - コストが高いとはいえない

構造化

- プログラムの内容を， 機能ごとに切り分ける
 - 一般には関数化
 - よく使われる動作をまとめるため
 - 機能を分けるため
 - プログラム内で一度しか使われなくても
 - 機能が違うなら関数化
 - main関数はほとんど関数呼び出しだけ，が理想
 - ファイルを機能で分けて，分割コンパイル
- データの取り扱い（データ構造）も機能ごと
 - 一つの機能や項目の表現に，複数の変数が必要
→ 構造体でまとめる
 - オブジェクト指向対応言語→クラスを駆使
 - ヘッダファイルに記述

ゲーム制作での構造化（例）

・機能例

初期化处理

- ・システム初期化
 - ・マップ情報読込
 - ・キャラ情報設定
- ・UI初期化
 - ・ウインドウ生成
 - ・画像読み込み
 - ・テクスチャー生成

メイン処理

- ・入力（イベント）読取
 - ・キー，マウス
 - ・Joy-Con，ゲームパッド
 - ・ネットワーク通信
- ・ゲーム制御
 - ・動作更新（座標など）
 - ・当たり判定
- ・描画
 - ・マップ
 - ・キャラ，スコアなど

終了処理

- ・メモリ破棄（free()）
- ・テクスチャーなど破棄
- ・ウインドウ消去

・場面転換がある例

初期化

- ・システム
- ・UI（メインウインドウ）

タイトル

- ・初期化
- ↓
- ・メイン
- ↓
- ・終了

Stage1

- ・初期化
- ↓
- ・メイン
- ↓
- ・終了

Stage2

- ・初期化
- ↓
- ・メイン
- ↓
- ・終了

終了処理

- ・ウインドウ消去

ゲーム制作での構造化（例）

- データ構造の例

- システム情報

- ステージ
 - スコア
 - ウィンドウ, など

- キャラ情報

- 座標
 - 当たり判定の範囲
 - HP
 - 画像情報
 - アニメーションパターン, など

- マップ情報

- 配置
 - 画像情報, など

```
typedef struct {  
    int stage;  
    int score;  
    SDL_Window window;  
    SDL_Renderer render;  
    :  
} GameInfo;
```

```
typedef struct {  
    SDL_Point pos;  
    SDL_Rect mask;  
    int hp;  
    SDL_Texture *image;  
    int ani_pat ;  
    :  
} CharaInfo;
```

```
typedef struct {  
    int map[ MAP_Width ][ MAP_Height ];  
    SDL_Texture *bg;    // 背景  
    SDL_Texture *chip;  // マップチップ  
    :  
} MapInfo;
```


エラーハンドリング

- 異常(エラー)が起きても, 正常に継続・終了させる
 - 異常終了（落ちる）ことがあってはならない
 - エラーメッセージを表示
 - 継続可能なら継続
 - 無理なら終了処理
 - プログラム使用者が間違った操作をしても対処
 - `assert()`はバグ検出には有用だがエラー処理ではない
- エラーの処理方法
 - 関数の引数, 戻り値をチェックして, エラーの場合
 - 値の変更で継続可能なら, 適当な値に変更
 - ユーザ操作が原因の場合, 再度操作を促す
 - エラー処理ルーチンから終了処理へ遷移

エラーハンドリング例

値の変更で継続可能なら、適当な値に

```
// 年齢を入力し、入場料金を返す
int price( int age )
{
    int ret = 1000;
    // 引数のチェック
    if (age < 0){
        // 0歳としても差し支えない場合
        age = 0;
    }
    if( age < 4 ){
        ret = 0;
    }
    else if ( age < 13 || 60 <= age ){
        ret = 600;
    }
    return ret;
}
```

ユーザ操作が原因の場合、再度操作を

```
:
do {
    printf("年齢 : ");
    scanf( "%d", &age );
    // ユーザー操作
    // 入力値をチェック
    if( age < 0 ){
        puts("もう一度入力して" );
    }
} while( age < 0 );
:
```

エラーハンドリング例 (エラー処理ルーチンから終了処理へ)

- エラー処理ルーチン

- その場で用意した資源は片付ける
 - メモリは解放
 - ファイルは閉じる, など
- メッセージやログを表示してもよい

- 終了処理へ遷移

- 呼び出し元へ伝える
 - 通常エラーコードを用意
 - defineマクロなどを使う
- 呼び出し元でも同様にエラー処理
 - main()でも
 - main()のreturn値はターミナルへ返る (echo \$?)

```
FILE *fp=fopen("datafile","r");
int *map=(int *)malloc(MAP_Width
                        * MAP_Height * sizeof(int) );
if( !map ){
    fclose( fp ); return -1;
}
:
for( int y=0; y<MAP_Height; y++ ){
    for( int x=0; x<MAP_Width; x++ ){
        if( 1 != fscanf(fp, "%d",
                        &(map[y*MAP_Height + x])) ){
            fclose(fp); free(map);
            return -1;
        }
    }
}
:
fclose(fp); free(map);
return 0;
```

好まれないコーディング

- マジックナンバー

- ソース中に定数（数値）を直書きすること．その数値
 - 数値の意味はわからないがプログラムは正しく動く，まるで魔法の数値
- 他人が読んで，意味不明になるような数値には名前を付ける（意味を持たせる）
 - defineマクロ，enum(列挙定数)，const修飾などを駆使

```
// defineマクロの例
#define MAP_Width 60
```

```
// enumの例
enum {
    MSG_None = 0,
    MSG_GameOver = 1,
    MSG_Clear = 2
};
```

```
// constの例
const SDL_Color Blue = {0,0,255,255};
```

```
// 使用例
int map[ MAP_Width ] = {0};
```

```
switch(msg){
case MSG_GameOver:
    puts("Game Over"); break;
case MSG_Clear:
    puts("Clear"); break;
}
```

```
sf = TTF_RenderUTF8_Blended
(ttf, "Clear", Blue);
```

好まれないコーディング

- goto
 - プログラムの流れを強制的に変えることができるので、可読性が極めて悪くなる
 - ラベルを付ければ何処にでもジャンプできるため
 - 基本的には使わず、例外的な処理（エラーなど）にとどめる
 - 例外処理機構のある言語では例外処理機構を使う
 - 例外的（エラー）処理でのgoto使用例 

```
int ret=-1;
FILE *fp=fopen("datafile","r");
int *map=(int *)malloc(MAP_Width
                       * MAP_Height * sizeof(int) );
if( !map ){ goto CLOSEFREE; }
:
for( int y=0; y<MAP_Height; y++ ){
    for( int x=0; x<MAP_Width; x++ ){
        if( 1 != fscanf(fp, "%d",
                        &(map[y*MAP_Height + x])) ){
            goto CLOSEFREE;
        }
    }
}
:
ret=0;
CLOSEFREE:
fclose(fp); free(map);
return ret;
```